WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

仓颉兴趣组趣味项目分享

华为开发者布道师 东北大学软件学院 张引

TOML

Web Server

分布式运行时

云原生

DEVELOPER ADVOCATES



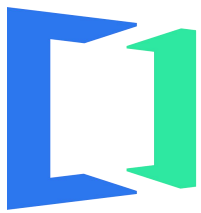
目录

- 关于仓颉兴趣组
- cangjie_toml
- tea with seedhttp
- Dapr Cangjie SDK
- naivetoolkit
- simplekv
- sftp
- learning_ml_cj

WORKSHOP 研讨/分享/感悟



=



+



仓颉 Cangjie

cangjie_toml

- cangjie_toml是一个符合仓颉语言标准库serialization包标准的toml文件解析库，支持toml文件到DataModel的转换
  - https://gitcode.com/PermissionDog/cangjie_toml
  - 孟世博，东北大学软件学院软件工程专业

cangjie_toml

```
private const toml = """
name = "SIGCANGJIE"
from = "neu"
"""
private class SigCangjie {
    public SigCangjie(
        public let name: String,
        public let from: String
    ) {}
}
extend SigCangjie <:
    Serializable<SigCangjie> { // ...
```

```
let sigcangjie = toml |>
    DataModel.fromToml |>
    SigCangjie.deserialize
```

cangjie_toml

```
let dm = DataModel.fromTomlFile("cjpm.toml")

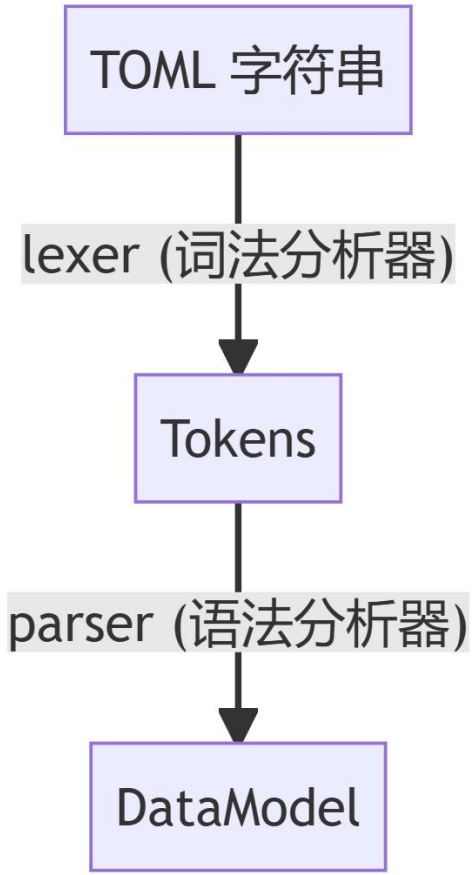
let root = (dm as DataModelStruct).getOrThrow()

let pkg = (root.get("package") as DataModelStruct).getOrThrow()

let cjcVersion = (pkg.get("cjc-version") as DataModelString).getOrThrow().getValue()

let name = (pkg.get("name") as DataModelString).getOrThrow().getValue()
```

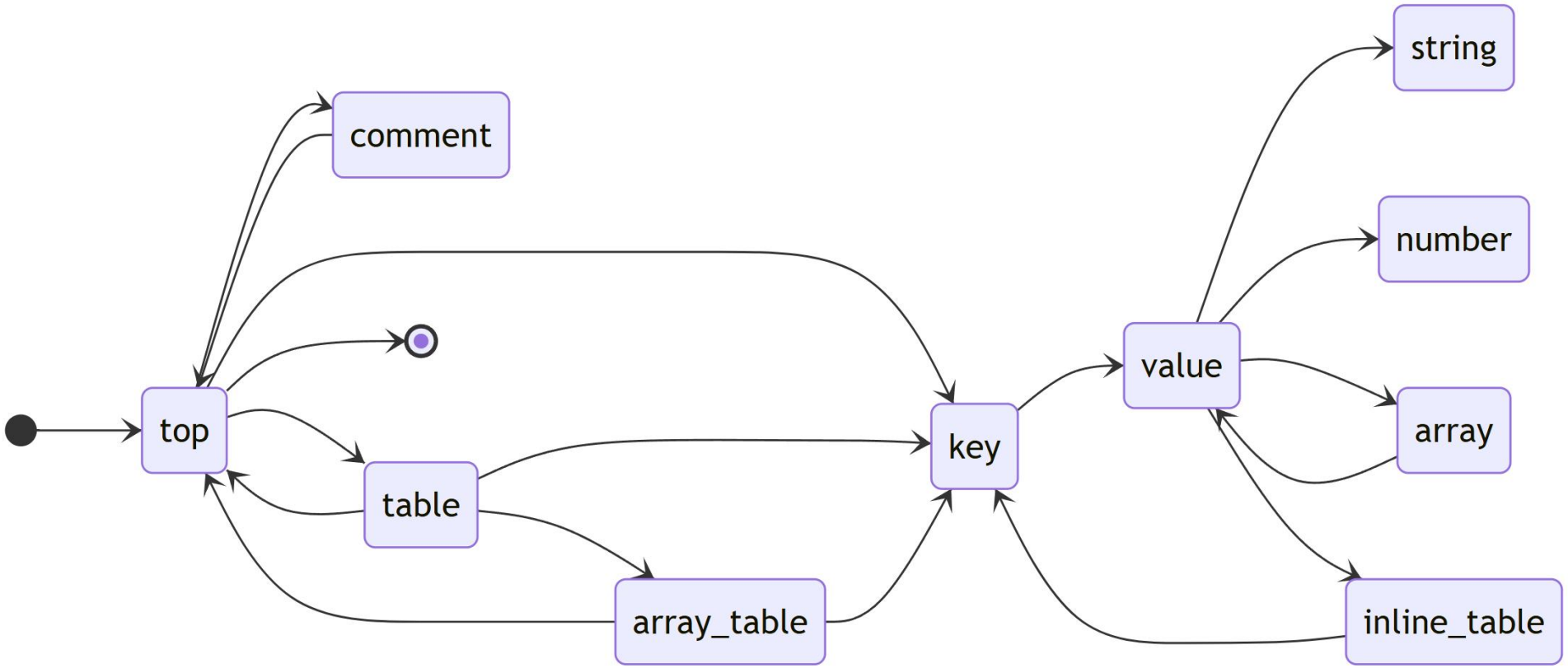
cangjie_toml关键技术



cangjie_toml关键技术

```
// 状态定义
enum State {
    // Func 表示该状态存在
    // 其中函数参数接收一个 Lexer 对象，返回下一个执行的状态。
    | Func((Lexer) -> State)
    // End 表示结束状态
    | End
}
```

cangjie_toml关键技术



cangjie_toml关键技术

```
func lexTop(lexer: Lexer): State {
    let r = lexer.next()
    if (isWhiteSpace(r) || isNL(r)) {
        return lexSkip(lexer, Func(lexTop))
    }
    match (r) {
        case r'#' =>
            lexer.push(Func(lexTop))
            return Func(lexCommentStart)
        case r'[' =>
            return Func(lexTableStart)
        case r'\0' =>
            if (lexer.pos > lexer.start) {
                return lexer.error(LexException("unexpected
EOF"))
            }
            lexer.emit(TokenType.EOF)
            return End
        // ...
```

cangjie_toml

- 孟世博 cangjie_toml
  - https://gitcode.com/PermissionDog/cangjie_toml

tea with seedhttp

- seedhttp 是一个专注于性能优化的底层 HTTP 库
- tea 是一个基于 seedhttp 构建的 Web 框架，提供了类似于 Express.js和Gin的简洁的 API，帮助开发者高效构建高性能 Web 应用
  - 精简、函数式、灵活、高效
  - https://gitcode.com/yishengTH/SeedHTTP
  - https://gitcode.com/yishengTH/tea
  - https://gitcode.com/yishengTH/TeaExample
  - 李智杰，兰州交通大学电子与信息工程学院软件工程专业

tea with seedhttp

```
let app = Tea.default()          // 1. 初始化Tea
app.get("/hello") { c =>         // 2. 注册简单路由
    c.string(200, "Hello Tea")
}
app.run(8080)                    // 3. 启动监听
```

tea with seedhttp

```
let app = Tea.default()
// 全局中间件：异常捕获 + 跨域
app.use(busRecoveryMiddleware, corsMiddleware)
// 无需登录的路由组
let publicGroup = app.group("public")
publicGroup.post("/register", registerHandler)
publicGroup.post("/login", loginHandler)
// TodoList
publicGroup.get("/todo", getTodoHandler)
publicGroup.get("/todos", listTodosHandler)
// 需要登录的路由组
let authGroup = app.group("auth")
authGroup.use(authMiddleware)
// TodoList
authGroup.post("/todo", createTodoHandler)
authGroup.put("/todo", updateTodoHandler)
authGroup.delete("/todo", deleteTodoHandler)

app.run(8080)
return 0
```

tea with seedhttp关键技术

- 框架主要分为框架部分和HTTP 核心部分，通过极少的接口相互联动
  - 核心部分：tea 使用 seedhttp 作为底层 HTTP 引擎，所有 HTTP 请求和响应的处理均由 seedhttp 完成。tea 将 seedhttp 的低级接口抽象为类似 Express.js、Gin的高层API，使开发者不需要操作复杂的seedhttp
  - 框架部分：提供路由、路由组 (group)、中间件 (middleware) 机制、上下文 (TeaContext) 等高级用法

- 使用压缩前缀树来解析路由，能在复杂路由场景下保持快速、高效的匹配
  - 通过对前缀进行分段压缩，大幅减少路由节点冗余
  - 支持带参数的路由（如 /user/:id），并在 TeaContext 中自动填充 params
  - 支持自动处理不匹配或方法不允许的情况 (404/405)

- 通过 RouterGroup 来实现“分组路由”的能力，可为同一组路由统一设置一些中间件
  - let authGroup = app.group("auth")
  - authGroup.use(authMiddleware) // 给该组添加鉴权中间件
  - authGroup.post("/todo", createTodoHandler)
  - authGroup.put("/todo", updateTodoHandler)

- TeaContext 是与请求对应的上下文对象，封装了请求/响应的各类操作
  - context.query("id") 获取查询参数
  - context.param("id") 获取路由参数
  - context.requestHeader("Auth") 获取请求头
  - context.bindJson<T>() 反序列化请求体为 T 类型
  - context.json(code, data) 以 JSON 方式写响应
  - context.string(code, data) 写字符串响应
  - context.set/get(key, value) 在中间件或路由之间共享数据

- 李智杰 tea with seedhttp
  - https://gitcode.com/yishengTH/SeedHTTP
  - https://gitcode.com/yishengTH/tea
  - https://gitcode.com/yishengTH/TeaExample







Dapr Cangjie SDK

- Dapr是一个分布式运行时，为微服务架构应用的构建提供了十余项便捷的功能模块，包括服务调用、发布/订阅、配置管理、分布式锁、密钥管理等
- Dapr Cangjie SDK是Dapr SDK的仓颉语言实现
  - https://gitcode.com/PermissionDog/dapr-cangjie-sdk
  - 孟世博，东北大学软件学院软件工程专业

Dapr Cangjie SDK

```
import express from 'express'
const app = express()
app.post('/add', (req, res) => {
    let nums: number[] = req.body.nums
    res.json({
        result: nums.reduce((a, b) => a + b, 0)
    })
})
```

```
let client = DaprHttpClientBuilder()
    .host("localhost")
    .port(3500)
    .build()
// 要调用的服务名称
let serviceName = "add-service"
// 要调用的方法
let method = "add"
let nums = [ 1, 2, 3 ]
```

```
// 调用服务
let res = client.invokeMethod(serviceName, method, Request
    .builder().post(RealRequestBody.create(
        MediaType.parse("application/json; charset=utf-8"),
        """
        {
            "nums": [${nums}]
        }
        """))
    .build())
let data = res.getBody().getOrThrow().getString()
```

```
// pubsub 名称
let pubsubName = "my-pubsub"
// 主题
let topic = "newOrder"
// 发送事件数据，由 dapr 封装为 CloudEvent
// 未指定事件类型，将被路由至 /otherorders
client.publishEvent(pubsubName, topic,
    HashMap<String, String>([
        ("foo", "bar")                  // 事件数据
    ]).serialize()
)
```

```
// 创建自定义 CloudEvent，指定事件类型为 order，将被路由至 /orders
let cloudEvent = CloudEventBuilder<HashMap<String, String>>()
    .id("A123-456-789")
    .subject("mysubject")
    .eventType("order")                 // 指定事件类型
    .source("mysource")
    .data(HashMap<String, String>([
        ("foo", "bar")                  // 事件数据
    ]))
    .build()
// 发送原始 CloudEvent 数据
client.publishEvent(pubsubName, topic,
    cloudEvent.serialize())
```

```
// 对应的 configuration store
let storeName = "store-name"
// 要查询的键
let keys = ["key1", "key2"]
// 得到查询到的配置信息
client.getConfiguration(storeName, keys)
```

```
// 对应的 lock store
let storeName = "store-name"
// 用于标识资源的 ID
let resourceId = "foo-bar"
// 标记所有者
let owner = "owner-name"
// 标记过期时间，单位为秒
let expiryInSeconds = 60
```

```
// 上锁
try {
    client.lock(storeName, resourceId, owner,
expiryInSeconds)
    println("上锁成功")
} catch (e: DaprException) {
    println("上锁失败")
}
// 解锁
client.unlock(storeName, resourceId, owner)
```

```
// 对应的 secret store
let storeName = "store-name"
// 要获取的键
let key = "ConnectionStrings:TextItemContext"
// 获取 secret
client.getSecret(storeName, "ConnectionStrings:TextItemContext")
```

```
{
    "ConnectionStrings:TextItemContext": "Server=recall-sqldata;Database=RecAll.TextListDb;User Id=sa;Password=Pass@word;TrustServerCertificate=true"
}
```

Dapr Cangjie SDK

- 孟世博 Dapr Cangjie SDK
  - https://gitcode.com/PermissionDog/dapr-cangjie-sdk



- naivetoolkit是一个简易的仓颉领域驱动设计云原生开发套件
  - naiveioc ： 一个不依赖反射和AOT的依赖注入容器
  - naivemediator ： 一个不依赖反射和AOT的中介者模式实现
  - navievalidation ： 一个简易的数据验证工具
  - navieseedwork ： 一个领域驱动设计基础设施套件
  - naiveserviceresult ： 一个通用返回值套件
  - naivesqlbuilder：一个简易的SQL生成工具
  - naiveteaset ： 将naivetoolkit集成到tea

- 一个简易的仓颉领域驱动设计云原生开发套件





```
serviceCollection = NaiveServiceCollection()
@NIAddScoped[serviceCollection](Engine)
@NIAddScoped[serviceCollection](ICar, Car)
let serviceProvider = serviceCollection.buildServiceProvider()

let scopedServiceProvider = serviceProvider.createScoped()
let car = @NIGetRequiredService[scopedServiceProvider](ICar)
```

```
let serviceCollection = NaiveServiceCollection()
@NIAddScoped[serviceCollection](Bag)
@NIAddScopedFactory[serviceCollection](Handle, {_:INaiveServiceProvider => Handle()})
let serviceProvider = serviceCollection.buildServiceProvider()

let scopedServiceProvider = serviceProvider.createScoped()
let handle1 = @NIGetRequiredService[scopedServiceProvider](Handle)
```

naiveioc

```
let serviceCollection = NaiveServiceCollection()
@NIAddSingleton[serviceCollection](Engine)
@NIAddSingleton[serviceCollection](ICar, Car)
@NIAddSingleton[serviceCollection](Logger0<>)
@NIAddSingleton[serviceCollection](Logger1<>)
@NIAddSingleton[serviceCollection](Logger2<>)
@NIAddSingleton[serviceCollection](Student)
@NIAddSingleton[serviceCollection](Class)
```

```
@NIService
public class Class <: Resource {
    public Class(public let student: Student,
        public let logger: Logger0<Logger2<Logger0<Student>, Logger0<Class>>>) {}

    private var _isClosed = false
    public func isClosed(): Bool {
        return _isClosed
    }
    public func close(): Unit {
        _isClosed = true
    }
}
```

naivemediator

```
let serviceCollection = NaiveServiceCollection()
@NIAddSingleton[serviceCollection](NaiveMediator)
@NIAddSingleton[serviceCollection](IPipelineBehavior, OuterLoggingBehavior)
@NIAddSingleton[serviceCollection](IPipelineBehavior, InnerLoggingBehavior)
@NIAddSingleton[serviceCollection](CreateTodoCommandHandler)
@NMRegisterHandler(CreateTodoCommand, CreateTodoCommandHandler)
let serviceProvider = serviceCollection.buildServiceProvider()

let mediator = @NIGetRequiredService[serviceProvider](NaiveMediator)
let request = CreateTodoCommand()
let response = mediator.send(request)
```

naivevalidation

```
public class PersonValidator <: AbstractValidator<Person> {
    public PersonValidator() {
        @NVRuleFor({p => p.forename}).must({p => p == "Jim"}).withMessage("forename must be Jim")
    }
}

let person = Person()
person.forename = "Allen"
let validator = PersonValidator()
let result = validator.validate(person)
```

naivesqlbuilder

```
let builder = @NSSelectFrom(DataModel) {p => [p.id]}.setWhere {p => p.id == 5}.and {p => p.name == ""}

let builder = @NSInsertInto(DataModel) {p => [p.id == 5, p.name == ""]}

let builder = @NSUpdate(DataModel) {p => [p.name == ""]}.setWhere {p => p.id == 5}
```

```
let serviceCollection = NaiveServiceCollection()
@NIAddSingleton[serviceCollection](NaiveMediator)
@NIAddSingleton[serviceCollection](IPipelineBehavior, ValidatorBehavior)
@NMRegisterSingletonHandler[serviceCollection](CreateTodoCommand, CreateTodoCommandHandler)
@NVRegisterSingletonValidator[serviceCollection](CreateTodoCommand, CreateTodoCommandValidator)
let serviceProvider = serviceCollection.buildServiceProvider()

let mediator = @NIGetRequiredService[serviceProvider](NaiveMediator)
let createTodoCommand = CreateTodoCommand()
createTodoCommand.request = "this"
```

```
main(): Int64 {
    let tea = Tea.default()
    tea.use(globalExceptionHandler)
    tea.useNaiveIoc {
        services => services |> useDaprClient |> addCustomApplicationServices |> addCustomMediator
    }
    tea |> mapHandlers
    tea.run(8080)
    return 0
}
```

- tea with seedhttp + naivetoolkit + Dapr Cangjie SDK
- 可能是国内首门“国产自主语言+高校自建框架”的领域驱动设计云原生开发课程
  - （此处要确保定语足够长）
- 全量无偿回馈给充满爱与希望的仓颉社区
  - 课程视频
  - 课程代码
  - 开发框架
  - 参考文档


naivetoolkit

simplekv

- simplekv是一个高效、简洁的键值存储库，支持快速的数据查找、插入、删除，并具备排序数据获取、范围扫描、并发安全等高级功能，适用于需要高性能数据存储的场景
  - https://gitcode.com/leaveWhite9088/simplekv
  - 唐梓鑫，东北大学软件学院软件工程专业

simplekv

```
// 配置
let config = SimpleKVConfig()
config.fileMode = "0777"
config.dirMode = "0777"

// 打开数据库
let F: String = "./testDir/testDb.db"
let db = SimpleKVManager.default.open(F, config)
```

```
// 设置键值对
db.set(1, true)

// 获取键值对
let v = db.getBool(1)

// 删除数据库文件
db.deleteFile()
```

```
// 升序检索
let res = db.keys<String>(None, 0, 0, true)
var s = ""
for(r in res){
    s += r.toString()
}
println(s)
```

```
// 升序偏移和限制
let reslimit  = db.keys<String>(None, 2, 2, true)
s = ""
for(r in reslimit){
    s += r.toString()
}
println(s)
```

```
// 从特定字节开始的降序检索
let resfromdesc = db.keys<String>("10", 2, 2, false)
s = ""
for(r in resfromdesc){
    s += r.toString()
}
println(s)
```

```
// 按前缀升序查询所有
let resprefasc2 = db.keys<String>("1*", 0, 0, true)
s = ""
for(r in resprefasc2){
    s += r.toString()
}
println(s)
```

- 唐梓鑫 simplekv
  - https://gitcode.com/leaveWhite9088/simplekv



sftp4cj

- sftp4cj：使用仓颉实现的sftp服务器与客户端
  - https://gitcode.com/joy7/sftp4cj
  - 张博林，东北大学软件学院信息安全专业

```
let port = 8000u16
let server = SftpServer(port)
// 启动服务器
spawn { server.start() }
// 等待服务器启动
sleep(Duration.second * 2)
// 创建并连接客户端
let client = SftpClient("127.0.0.1", port,
Duration.second * 5, 3u32)
client.connect()
```

```
let testFile = "test.txt"
// 创建并写入文件
let handle = client.openFile(testFile,
OpenFlags.WRITE | OpenFlags.CREATE |
OpenFlags.TRUNCATE, 0u32)
let testData = "Hello, SFTP!".toArray()
client.writeFile(handle, 0u64, testData)
client.closeFile(handle)
// 创建目录
client.mkDir("test_directory")
```

```
let readHandle = client.openFile("large_file.txt",
OpenFlags.READ, 0u32)

var offset = 0u64
var totalBytesRead = 0
let chunkSize = 8192u32  // 8KB chunks
```

```
while (totalBytesRead < 1024 * 1024) {
let data = client.readFile(readHandle, offset,
chunkSize)
let bytesRead = data.size


if (bytesRead == 0) {
break
}


totalBytesRead += bytesRead
offset += UInt64(bytesRead)
}
```

- 张博林 sftp4cj
  - 如果有工作机会，请联系 2890594946@qq.com
  - https://gitcode.com/joy7/sftp4cj



learning_ml_cj

- learning_ml_cj是一个大模型推理框架，支持开源的safetensor存储结构，提供基于CPU的大模型推理服务
  - https://gitcode.com/abcd1234-wyj/learning_ml_cj
  - 王翊嘉，东北大学软件学院软件工程专业

```
// “Once upon a time”
let input: Array<UInt32> = [1, 80, 147, 201, 282, 57];
// load the model
let model = Llama.from_example_files(Option<Path>.None);
// generate the next sentence
let output = model.generate(input, 200, 0.9, 40, –1.0);
let translator = Translator.from_example_files(Option<Path>.None);
let user_in = translator.translate(input);
let result = translator.translate(output);
println(user_in + result);
```

- 王翊嘉 learning_ml_cj
  - https://gitcode.com/abcd1234-wyj/learning_ml_cj



WORKSHOP 研讨/分享/感悟

仓颉社区中的灵感碰撞 / 社区先行者的干货分享

仓颉 Cangjie

感谢您的观看!

← 仓颉兴趣组首页，恳请指导
http://www.sigcangjie.cn

